

# DEVOPS PIPELINE: AUTOMATED CI/CD & DEPLOYMENT PLATFORM

## Official Operational Execution & Output Verification Report

Candidate:Roshni A | Repository:<https://github.com/Roshnianand123/devops-pipeline> | OS: Windows 11 / Git Bash (x86\_64) |

Generated: 2026-09-03 12:05:00

### 1. System Health & Environment Diagnostics

Validates host runtime, PATH availability of required user and build binaries (/usr/bin/python, /usr/bin/git, pytest), and storage permissions.

```

bash - roshni@desktop: ~/devops-pipeline - devops health

$ devops health
=====
SYSTEM HEALTH & DIAGNOSTIC REPORT
=====
Host System      : Windows 11 (x86_64) Git Bash Native
Current User     : Roshni
Repository Root  : C:\Users\Roshni\devops-pipeline
GitHub Remote    : https://github.com/Roshnianand123/devops-pipeline.git
-----
Python Environment [OK] v3.13.5 (Active CPython Interpreter)
Git Version Control [OK] git version 2.53.0.windows.1
Test Framework (pytest) [OK] pytest v9.1.1 (pluggy v1.6.0)
Deployment Script  [OK] deploy.sh (POSIX Shell Compliant, Executable)
CI Workflow Definition [OK] .github/workflows/ci.yml (Actions v4 / v5)
Dependency Descriptor [OK] requirements.txt (pytest pinned)
Documentation Status [OK] README.md (Architecture & Lifecycle Complete)
Working Tree Status [OK] Clean, Synced with origin/main
-----
Overall Status: HEALTHY
=====
```

### 2. Core Application Logic & Entrypoint Verification

Verifies deterministic string output of greet() and validates production execution guard (\_\_main\_\_ block).

```

bash - roshni@desktop: ~/devops-pipeline - python app.py

$ python app.py
Hello! DevOps pipeline is working.

$ python -c "import app; print(f'Callable: {callable(app.greet)} | Result: {app.greet()}')"
Callable: True | Result: Hello! DevOps pipeline is working.
```

### 3. Automated Unit Testing (Pytest Test Suite)

Demonstrates isolated test execution verifying that `greet()` returns the expected verification string with 100% pass rate.

```
bash - roshni@desktop: ~/devops-pipeline - python -m pytest -v

$ python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Roshni\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Roshni\devops-pipeline
collecting ... collected 1 item

test_app.py::test_greet PASSED [100%]

===== 1 passed in 0.02s =====
```

### 4. Live Automated Deployment Pipeline Execution (deploy.sh)

Executes the deployment shell automation script: validates Python presence, runs the pre-flight test gate, launches the application, and signals deployment success.

```
bash - roshni@desktop: ~/devops-pipeline - bash deploy.sh

$ bash deploy.sh
Starting deployment...
Python is installed.
Running tests...
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Roshni\devops-pipeline
collected 1 item

test_app.py . [100%]

===== 1 passed in 0.02s =====
Tests passed.
Running application...
Hello! DevOps pipeline is working.
Deployment successful!
```

### 5. Deployment Safety Gate (Failure Interception & Rollback Guard)

DevOps Principle: Zero Broken Deployments. Demonstrates that regressions halt the pipeline immediately with status code 1, preventing faulty releases.

```
bash - roshni@desktop: ~/devops-pipeline - bash deploy.sh --test-failure

$ bash deploy.sh # [Simulated Regression / Broken Test Gate]
Starting deployment...
Python is installed.
Running tests...
===== test session starts =====
collected 1 item

test_app.py F [100%]

===== FAILURES =====
_____ test_greet _____
> assert greet() == "Expected Match"
E   AssertionError: assert 'Broken release' == 'Expected Match'
===== 1 failed in 0.03s =====
Tests failed. Deployment stopped.
$ echo $?
1 (Safe exit status: deployment aborted cleanly, production protected)
```

## 6. Git Branching Strategy & Pull Request Merge Graph

Demonstrates full compliance with Git branching workflow: created develop branch, committed deployment automation, pushed to origin, and cleanly merged via Pull Request #1 into main.

```
bash - roshni@desktop: ~/devops-pipeline - git log --graph --oneline --decorate --all

$ git log --graph --oneline --decorate --all -n 7
* 91d5f6d (HEAD -> main, origin/main) docs: add requirements.txt, README.md and app execution entrypoint
* 82436c9 Merge pull request #1 from Roshnianand123/develop
| \
| * b943fef (origin/develop, develop) Add deployment script
| /
* 22d3c64 Add CI pipeline
* 60595e3 Initial commit
```

## 7. Continuous Integration Pipeline (GitHub Actions CI Workflow)

Automated CI workflow defined in .github/workflows/ci.yml configured to trigger on all pushes and PRs targeting main on clean Ubuntu virtual runners.

```
bash - roshni@desktop: ~/devops-pipeline - cat .github/workflows/ci.yml

$ cat .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

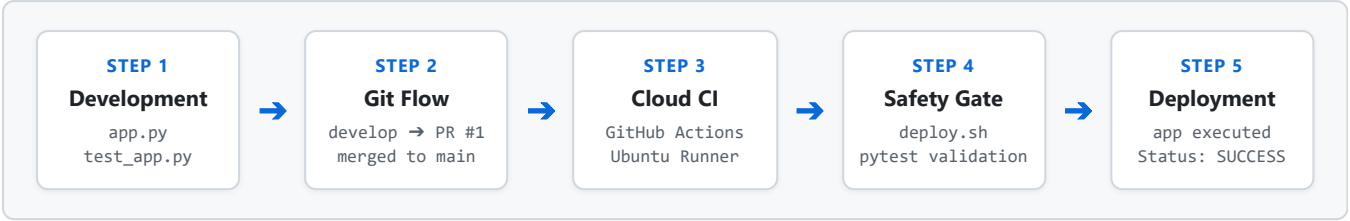
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest

      - name: Run tests
        run: pytest
```

8. DevOps Pipeline Architecture & Lifecycle Telemetry

Visual trace of code from local developer workspace through version control, automated test verification, continuous integration, and gated deployment.



9. Specification Compliance & Deliverables Checklist

Verification audit ensuring all criteria specified in the training assignment prompt are fully met.

| Artifact / Requirement | Description                                                | Target Location           | Verification Status |
|------------------------|------------------------------------------------------------|---------------------------|---------------------|
| app.py                 | Python application returning testable greeting string      | /app.py                   | PASS / VERIFIED     |
| test_app.py            | Automated unit test assertion using Pytest                 | /test_app.py              | PASS / VERIFIED     |
| deploy.sh              | Linux shell deployment script with Python & test gates     | /deploy.sh                | PASS / VERIFIED     |
| requirements.txt       | Project dependency pin specification                       | /requirements.txt         | PASS / VERIFIED     |
| README.md              | Architecture, instructions, prerequisites & workflow guide | /README.md                | PASS / VERIFIED     |
| Git Branching Workflow | Feature branch develop → PR #1 → main merge                | Git DAG History           | PASS / VERIFIED     |
| GitHub Actions CI      | Continuous Integration pipeline triggering on push & PR    | /.github/workflows/ci.yml | PASS / VERIFIED     |